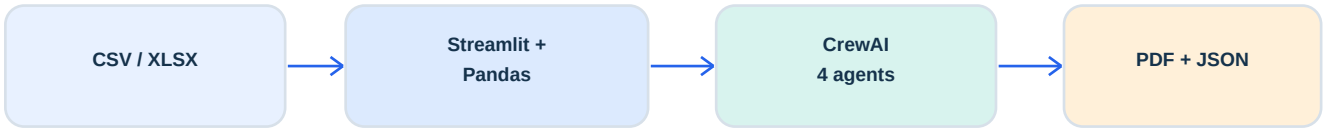
ASSETMIND

Design, Development, Deployment and Evaluation of a CrewAI-Based IT Asset Report Generator

Project Report for Mentor Review



Evidence-grounded analysis; numeric totals remain deterministic

Project	AssetMind — Multi-Agent IT Asset Report Generator
Prepared for	Project Mentor
Report date	24 October 2026
Application	Streamlit, Pandas, CrewAI, ReportLab
AI provider	Google Gemini (optional; live provider execution not included in evaluation)

Evidence note. This report describes the implemented prototype and measured local checks. It does not claim production enterprise readiness, organizational savings, or successful live Gemini inference.

Report guide

- 1. Abstract and project overview
- 2. Problem definition, objectives, and scope
- 3. System design and multi-agent workflow
- 4. Development and implementation
- 5. Deployment and operation
- 6. Evaluation factors, measured results, and interpretation
- 7. Limitations, privacy, and future work
- 8. Conclusion and references

1. Abstract and project overview

AssetMind is an academic prototype for turning an IT asset inventory spreadsheet into a verified dashboard and a reviewable report. It combines deterministic Pandas calculations with four distinct CrewAI roles: data-quality analyst, analytics specialist, enterprise report writer, and report quality reviewer. The Streamlit interface accepts CSV and XLSX, preserves input rows and unknown columns, shows validation and charts, and exports PDF and JSON. A rule-based Asset Attention Radar prioritizes records for human follow-up with visible evidence and suggested next steps.

The prototype was validated with 29 offline tests and a synthetic reference fixture. All fixture comparisons passed. A live public Streamlit page was confirmed to load. No real Gemini generation was run during evaluation; therefore agent quality, provider reliability, cost, and end-to-end LLM-backed completion are explicitly unmeasured.

2. Problem definition, objectives, and scope

2.1 Problem definition

IT asset reports commonly begin as spreadsheets with inconsistent column names, missing identifiers, duplicate records, inconsistent status/boolean representations, and incomplete assignment or warranty data. Manual consolidation takes time and makes it easy to misstate inventory totals or overlook records requiring follow-up. Generative AI can improve the readability of findings, but must not be allowed to invent or override authoritative counts.

Problem statement: How can a lightweight, locally runnable prototype validate an uploaded IT asset inventory, calculate reproducible statistics, and produce useful narrative reporting while making uncertainty and unsupported AI claims visible?

2.2 Objectives

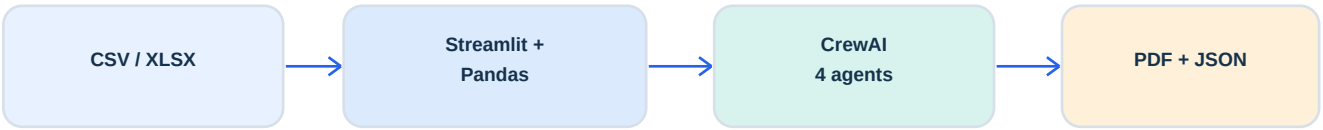
- Accept CSV/XLSX inventory files, preview data, normalize recognized headers, and report validation issues.
- Compute allocation, retirement, missing-value, duplicate-ID, and group-distribution results deterministically.
- Use exactly four sequential CrewAI agents for evidence-based analysis, writing, and review.
- Create downloadable PDF and JSON reports, including a deterministic, explainable Attention Radar.
- Evaluate deterministic calculations against a synthetic expected-results fixture and test failure paths without a paid LLM.

2.3 Scope and non-goals

The deliverable is a single-process academic prototype, not a CMDB, endpoint-management tool, financial valuation system, risk-scoring product, or production enterprise service. It has no authentication, persistent database, background workers, or cloud data warehouse. The sample data is fictional; no internship outcomes or company results are claimed.

3. System design and multi-agent workflow

3.1 Architecture



Evidence-grounded analysis; numeric totals remain deterministic

A single Streamlit process handles upload and interface logic. A dedicated data-processing module loads and validates files. Pandas analytics calculates verified totals and distributions; that evidence is shown to the user and supplied to CrewAI. ReportLab creates a PDF in memory and the app offers JSON serialization. The project has no server-side database or separate API.

3.2 Data definitions and ambiguity

Area	Prototype rule
Identifier	Asset Tracking ID (or an unambiguous recognized equivalent); ambiguous mappings are reported, not guessed.
Allocated	A non-empty Allocated To field; if the column is absent, allocation is unknown rather than unallocated.
Retired	Explicit true isRetired or a retired/disposed/decommissioned status; retired takes precedence.
Active	Explicit false isRetired or active/in-service/deployed/in-use status, unless retired.
Unknown lifecycle	Records with no recognized active or retired evidence stay unknown.
Duplicate IDs	Trimmed, case-insensitive duplicate values are reported; source rows are not silently removed.
Data-quality score	Percentage of source rows with no blank/normalized-unknown cells across provided fields; a completeness indicator only.

3.3 Four-agent CrewAI sequence

Agent	Responsibility and boundary
1. Asset Data Quality Analyst	Explains supplied missing/duplicate/validation findings and practical corrections; cannot invent validation results.
2. IT Asset Analytics Specialist	Interprets verified counts/distributions; distinguishes observations from hypotheses.
3. Enterprise Report Writer	Produces a structured executive summary, findings, and prioritized recommendations from prior task context.
4. Report Quality Reviewer	Checks required sections and evidence support, flags unsupported claims, and can trigger at most one bounded revision.

CrewAI uses sequential orchestration and Pydantic output models. A separate numeric-claim check compares explicitly named metric statements with deterministic analytics. If provider setup or execution fails, the app retains the deterministic results and reports the AI failure instead of substituting invented prose.

3.4 Distinctive feature: Asset Attention Radar

The Radar is a deterministic review-prioritization feature for human triage. It assigns transparent points for missing/duplicate identifiers, explicitly flagged zombie assets, retired-but-assigned records, conflicting retirement fields, expired or soon-due warranties, and selected missing core fields. Scores are capped at 100; thresholds are Urgent (70+), High (40–69), Watch (20–39), and Routine (<20). The dashboard shows up to 25 highest-scoring records with source row, evidence, and a suggested next step. Warranty checks use the report date. Unknown booleans/dates are not presumed true/expired.

Radar scores prioritize a review; they are not estimates of financial, cyber-security, or hardware-failure risk. For the synthetic sample evaluated on 24 October 2026, 48 records crossed the 20-point threshold (3 Urgent, 20 High, 25 Watch). This is an observed sample feature result, not a benchmark or forecast.

4. Development and implementation

Technology	Use in the prototype
Python	Application, orchestration, analytics and evaluation scripts.
Streamlit	Complete interface, file upload, tabs, status, charts, and downloads.
Pandas / OpenPyXL	CSV/XLSX parsing, normalization, validation, grouping, deterministic metrics.
CrewAI / Google Gen AI	Sequential four-agent workflow using Gemini when configured.
Pydantic	Structured agent result schemas and validation.
ReportLab	In-memory, multi-section PDF report generation.
Plotly	Dashboard charts for category, department, and Radar priority.
pytest	Offline automated tests with provider calls stubbed/mocked.

4.1 Processing and output modules

File	Purpose
app.py	Streamlit interface, session state, visualizations, CrewAI invocation, download actions.
src/data_processor.py	CSV/XLSX loading, alias normalization, required-field checks, duplicates, booleans.
src/analytics.py	Deterministic inventory metrics, grouped counts, numeric-claim audit, Attention Radar.
src/agents.py	Gemini configuration, four CrewAI agents/tasks, structured outputs, bounded review.
src/schemas.py	Pydantic finding, analysis, recommendation, and reviewer models.
src/report_generator.py	In-memory PDF sections, tables, evidence and limitations.
evaluation/evaluate.py	Reproducible Python-only fixture evaluation; optional authorized real-LLM comparison.
tests/	Offline data processing, analytics, agent-boundary, and PDF tests.

4.2 Data handling and privacy

The upload limit is 10 MB; extension/content errors are surfaced. Source columns and rows are preserved for review. Full source rows are not passed as agent evidence; the Gemini task receives the calculated analytics object, source filename, and aggregate Radar information. That analytics object can include duplicate identifier values, so it is not a guarantee that all identifiers are excluded from the prompt. Use synthetic or authorized, appropriately minimized data when calling an external provider. The sample fixture contains fictional values only. API credentials belong in local .env or Streamlit Secrets, never Git.

Current local setup uses GEMINI_API_KEY and optional GEMINI_MODEL (default gemini/gemini-2.5-flash). The deployed page and deterministic workflow were verified, but a real Gemini call was not performed for the reported evaluation.

5. Deployment and operation

Deployment factor	Verified project state
Source repository	Public GitHub repository: github.com/pranavvwhoo/assetmind (main branch).
Hosted app	https://assetmind-ai-pranavvwhoo.streamlit.app/
Entrypoint	app.py
Hosting	Streamlit Community Cloud; app page was opened and rendered successfully.
Secrets	GEMINI_API_KEY must be set in Streamlit app Secrets for real Gemini-backed generation.
Persistent data	None; report history is Streamlit session state and resets with the session.
App name note	Requested assetmind-ai subdomain was already taken; available owner-specific slug was deployed.

5.1 Local Windows PowerShell reproduction

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
# Set GEMINI_API_KEY in .env only if AI generation is needed
streamlit run app.py
```

To reproduce the offline checks, run `python -m pytest -q` and `python -m evaluation.evaluate`. An actual hosted deployment does not imply the Gemini secret has been configured or that external inference has succeeded.

6. Evaluation factors, measured results, and interpretation

6.1 Evaluation design

Evaluation separates deterministic correctness from LLM quality. The deterministic baseline is tested against a reviewed expected-results fixture for the 57-row fictional sample. Unit tests exercise file formats and failure cases, exact inventory definitions, duplicate metrics, Radar evidence/threshold behavior, PDF bytes, provider configuration, structured four-task orchestration, and numeric-claim review. CrewAI calls are mocked in unit tests; tests do not require a paid API.

Measure	Expected	Observed	Result
Asset rows	57	57	Pass
Allocated / unallocated	34 / 23	34 / 23	Pass
Retired / active / lifecycle unknown	5 / 46 / 6	5 / 46 / 6	Pass
Rows with missing information	30	30	Pass
Row completeness score	47.4%	47.4%	Pass
Missing identifier rows	1	1	Pass
Duplicate identifier values	1	1	Pass
Duplicate detection precision / recall	1.0 / 1.0	1.0 / 1.0	Pass
Per-column missing-value comparisons	All fixture columns	All matched	Pass
Offline automated tests	All pass	29 passed	Pass

The local reference evaluation completed in approximately 0.54 seconds in the recorded run. This is one machine-specific deterministic-script observation, not a performance benchmark or hosted response-time guarantee.

6.2 What these results establish—and do not establish

- Established: the tested deterministic totals, grouping/validation fixture comparisons, duplicate precision/recall, PDF generation, and Streamlit startup path worked in this environment.
- Not established: real Gemini response quality, live four-agent success rate, provider latency/cost, recommendation usefulness to asset owners, or behavior on representative enterprise inventories.
- The expected-results fixture is synthetic and small; full production-grade external validation has not been performed.
- A human-reviewed recommendation-quality benchmark is still required before drawing conclusions about AI benefit.

7. Limitations, privacy, and future work

Limitation	Implication / responsible next step
Synthetic reference data	Results demonstrate logic, not real organization-wide accuracy; obtain authorized anonymized validation data.
Rule-based lifecycle interpretation	Status vocabulary/field meaning vary; confirm mappings with inventory owners before operational use.
Attention Radar heuristic	Scores are arbitrary transparent review points; do not treat them as predicted risk or automate disposition.
No Gemini evaluation run	Set a key and perform an authorized run; log completion, duration, reviewer status, and human quality ratings.
Prompt includes analytics and duplicate ID values	Minimize/redact identifiers and obtain authorization before external-provider use; document retention/processing policy.
No authentication or persistent storage	Avoid confidential or multi-user production deployment without a designed access/data-governance layer.
Provider cost/usage unavailable	Add reliable usage/cost measurement before making cost-effectiveness claims.

Future evaluation and enhancement plan

- Ask a mentor or asset-management reviewer to inspect sample Radar cases and agree on whether the signals/actions are useful.
- Create an anonymized, manually adjudicated validation set; report field mapping accuracy and recommendation quality with a documented rubric.
- Add opt-in real Gemini evaluation only with authorized credentials; compare AI-assisted narrative with a deterministic-only baseline.
- Measure provider latency, completion/failure rate, and cost where reliable usage data is available; report multiple runs and conditions.
- For any broader deployment, design access controls, retention, audit logging, consent/authorization, and secure handling before uploading real inventory data.

8. Conclusion

AssetMind demonstrates a focused end-to-end prototype: spreadsheet upload, transparent deterministic analytics, four-role CrewAI orchestration, structured report review, a distinctive explainable attention queue, and PDF/JSON outputs. The measured synthetic fixture and 29 offline tests passed, and a public Streamlit deployment is available. These results support technical feasibility for the prototype—not enterprise readiness or verified AI value. A real-provider study and human review on authorized data are the key next evaluation steps.

References and project links

- Project source (public GitHub) — https://github.com/pranavvwhoo/assetmind
- Live Streamlit app — https://assetmind-ai-pranavvwhoo.streamlit.app/
- CrewAI documentation — https://docs.crewai.com/
- Streamlit documentation — https://docs.streamlit.io/
- Pandas documentation — https://pandas.pydata.org/docs/
- Pydantic documentation — https://docs.pydantic.dev/
- ReportLab documentation — https://www.reportlab.com/docs/reportlab-userguide.pdf
- Google AI Studio API keys — https://aistudio.google.com/app/apikey

Generated from the implemented project files and local test/evaluation outputs. Prepared for mentor review; institution-specific formatting and citation style may be applied before final submission.